Lab 04: Data Formats: XML and JSON

INFO 4617/5617 · Week 4 · Chapter 4, Data Formats: XML and JSON

This week every class is live coding: one lab in three parts, one a day. Each step says what we're building and why. Its code cells hold a comment saying what to write, and we write the code together. Cells that start with `# Given:` are already written: we read them together and run them. Nothing here is turned in. After each step, a 💬 **Discuss** box holds questions to talk over; click it to open it.

We read one real file in each format:

- **Part 1, Monday:** the roster of the U.S. House of Representatives, an XML file the Office of the Clerk publishes. We look at it before we parse it, read one member by hand, and count them all.
- **Part 2, Wednesday:** every member as one row of a table, and JSON as Python's lists and dictionaries.
- **Part 3, Friday:** a year of Boulder's daily weather from NASA's POWER API, as JSON, opened one key at a time, and checked against the place we meant.

The chapter's own JSON case, the Open-Meteo forecast, and an RSS feed are yours in the Recommended Exercises. Along the way, short ⏱️ microsprints ask you to change one thing, alone or in pairs, and share what you find.

Before class: open this notebook in the `webdata` environment (chapter 1), and have your email address ready for the User-Agent. Nothing this week needs a key or an account. Keep a browser open beside the notebook: we look at both files in it.

On Wednesday and Friday, start at that day's setup step.

> **Instructor key.** Rebuilt from scratch on 2026-11-11 for the next offering, in three parts, from chapter 4's Learning Objectives, after the audit in issue 178; it follows the textbook at commit `3036573`. Run on 2026-11-11 at 06:47 UTC with `labs/tools/run_key.py`, in Python 3.14.6 (beautifulsoup4 4.15.0, lxml 6.1.3, pandas 3.1.1, requests 2.34.2): every cell ran live, and none is tagged `not-run-live`. Five cells stop with an error on purpose (the misspelled tag in Step 3, the refusal in Step 12, and the three broken strings in Step 10's microsprint); they are tagged `raises-exception`, so a run goes on past them. The run asked the Clerk's site once, for the roster published November 3, 2026, and POWER's API six times. The saved copies in `labs/week-04/data/` were written by this run's own cells: the roster exactly as the Clerk sent it (saved as `MemberData.xml`, and renamed `MemberData-2026-11-03.xml`), `lab04_roster.csv`, `lab04_boulder_2025.json` and `lab04_places.csv`. For the run, `data_base` pointed at a local server for that folder; it was then set back to the repository's address on `main`, which works once this merges. A rerun should match every count from the saved roster (441 entries; R 221, D 217, two vacancies, one `I` whose caucus is `R`; five delegates and one resident commissioner; 13 footnotes; 29 of 44 states split; five garbled names) and every figure for 2025 (Boulder's grid square 19.7 °C and 115 nights below 0 °C, Leadville's 8.1 °C and 252, the point at 40.01, 105.27 14.8 °C and 171, Grand Junction's 18.9 °C and 122). It won't match the live roster in Steps 1 and 2 once the Clerk publishes a new file, as it will when a new Congress begins on January 3, 2027; the microsprint's most recent days in Step 14; or Step 10's messages on a Python older than 3.13.

Part 1 — Monday: a file is a tree

The House roster is one XML file: tags inside tags, like folders inside folders. Today we look at it before we parse it, read one member by hand, count every member, and then read the entries that make the count look odd.

Step 0 — Set up (4 minutes)

Put your email address in the User-Agent, and run the given cell. It imports what today needs, and names two addresses: the roster's, and the folder where the course keeps this week's saved copies.

To **parse** a file is to read its text and turn it into something Python can search. BeautifulSoup does that; its `"xml"` parser comes from the `lxml` package in `webdata`. If a cell stops with `bs4.FeatureNotFound`, the chapter's "Parsing with BeautifulSoup" says how to add it.

```
# Given: Part 1's setup. Put your email address in HEADERS, then run it
from collections import Counter
from pathlib import Path

import requests
from bs4 import BeautifulSoup

HEADERS = {"User-Agent": "WebDataScience/1.0 (your-email@colorado.edu)"}
roster_url = "https://clerk.house.gov/xml/lists/MemberData.xml"
data_base =
 ↪  "https://raw.githubusercontent.com/cuinfoscience/INFO4617-Fall2026/main/labs/week-04/data/"
```

💬 Discuss

- In week 2 we wrote this User-Agent. Who reads it, and what can they do with the email address in it?
- Neither site we use this week refuses a request without one. Why send it anyway?

Step 1 — Fetch the roster, and look before you parse (8 minutes)

Open the roster's address, https://clerk.house.gov/xml/lists/MemberData.xml, in your browser: Chrome draws an XML file that has no style information as an outline you can fold. Then fetch it, and look before parsing: the status code; the `Content-Type` header, the server's label for the kind of file; and the first bytes of `.content`, the file exactly as sent, which Python prints as `b'…'`. The server names no alphabet, so `requests` guesses one, `.encoding`.

```
# roster_response: requests.get() the roster_url, with headers=HEADERS
roster_response = requests.get(roster_url, headers=HEADERS)
```

```
# Print its status code, and roster_response.headers["Content-Type"]
print(roster_response.status_code, roster_response.headers["Content-Type"])
```

```
200 text/xml
```

```
# Print the first 150 bytes of roster_response.content. Find encoding= in its first line
print(roster_response.content[:150])
```

```
b'<?xml version="1.0" encoding="UTF-8"?><MemberData publish-date="November 3,
 ↪  2026"><title-info><congress-num>119</congress-num><congress-text>One Hundr'
```

```
# Print roster_response.encoding, the alphabet requests guessed. Does it agree?
print(roster_response.encoding)
```

```
ISO-8859-1
```

💬 Discuss

- When this key was written, `clerk.house.gov/robots.txt` answered 404. What did week 2 say a missing `robots.txt` means, and what doesn't it mean?
- If the Clerk had sent a 503 and an HTML error page, what would these three prints have shown?
- Your Recommended Exercises start the same way, with an RSS feed. What would you check here before writing any parsing code?

Step 2 — Parse it, and keep a saved copy (6 minutes)

`BeautifulSoup(roster_response.content, "xml")` parses the bytes. Give it the bytes, not `.text`, so it reads the encoding the file declares. `.find("MemberData")` returns the root tag, and its `publish-date` is an **attribute**, a `name="value"` pair inside the opening tag, read with square brackets, as in a dictionary. The roster changes as members come and go, so a given cell loads the course's saved copy into `soup`: everyone in the room, and next term, counts the same House. See "Working with Real XML".

```
# live_soup: parse roster_response.content with BeautifulSoup and the "xml" parser
live_soup = BeautifulSoup(roster_response.content, "xml")
```

```
# Print the "publish-date" attribute of live_soup's "MemberData" tag
print(live_soup.find("MemberData")["publish-date"])
```

```
November 3, 2026
```

```
# Given: save the file as it came, beside this notebook. It prints the number of bytes
Path("MemberData.xml").write_bytes(roster_response.content)
```

```
557204
```

```
# Given: the course's copy of the roster, saved by the key's run on 2026-11-11.
# Every cell from here on reads soup, this copy, so that everyone counts the same House
saved_response = requests.get(data_base + "MemberData-2026-11-03.xml", headers=HEADERS)
soup = BeautifulSoup(saved_response.content, "xml")
print(soup.find("MemberData")["publish-date"])
```

```
November 3, 2026
```

💬 Discuss

- In the key's run, the live roster and the saved copy were both published November 3, 2026. When will they differ, and which would you cite in a paper?
- What can a saved copy never tell you that the live request can?
- Week 3 said a table collected once is a snapshot, so record the date. What does `publish-date` let a reader of your analysis do that the counts alone don't?

Step 3 — One member by hand (14 minutes)

`.find()` returns the first tag with that name anywhere inside, however deep, and `.text` reads the words between its opening and closing tags. If no tag has that name, `.find()` returns `None`, Python's "nothing", and asking `None` for `.text` stops with an error. Names must match exactly, capitals included. The first member's state is `<state postal-code="AK"><state-fullname>Alaska</state-fullname></state>`: `AK` is an attribute, read with brackets; "Alaska" is text. See "Parsing with BeautifulSoup".

```
# first: the first "member" tag in soup, with soup.find()
first = soup.find("member")
```

```
# Print the text of first's "statedistrict" tag
print(first.find("statedistrict").text)
```

```
AK00
```

```
# Print the text of its "lastname" tag, then of its "party" tag
print(first.find("lastname").text)
print(first.find("party").text)
```

```
Begich
R
```

```
# state_tag: first's "state" tag. Print its "postal-code" attribute, with square brackets
state_tag = first.find("state")
print(state_tag["postal-code"])
```

```
AK
```

```
# Now print state_tag.text. Is it the postal code?
print(state_tag.text)
```

```
Alaska
```

```
# Misspell a tag on purpose: print first.find("lastName"), with a capital N
print(first.find("lastName"))
```

```
None
```

```
# Now print first.find("lastName").text, and read the last line of the error
print(first.find("lastName").text)
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[16], line 2
      1 # Now print first.find("lastName").text, and read the last line of the error
----> 2 print(first.find("lastName").text)

AttributeError: 'NoneType' object has no attribute 'text'
```

⏱️ Microsprint (4 minutes, alone or in pairs)

Do: the roster records each member's `<party>`, and also a `<caucus>`. Look up `<caucus>` in the Clerk's User Guide and Data Dictionary (page 4, as printed), then print the first member's caucus and district. **Share:** what the guide says `<caucus>` means, and the first member's caucus and district.

```
# Microsprint: print the text of first's "caucus" tag, then of its "district" tag
print(first.find("caucus").text)
print(first.find("district").text)
```

```
R
At Large
```

💬 Discuss

- `print(state_tag.text)` gave "Alaska" with no error. Which mistake is worse for a dataset: one that stops with an error, or one that doesn't?
- The roster keeps the postal code in an attribute and the full name in a child tag. If you designed the file, which would you choose, and why?
- The chapter also reaches tags with dots, `soup.members.member`. Why does `soup.member-info` stop with a `NameError`, and what does Python think you wrote?

Step 4 — Every member, and a count (10 minutes)

`.find_all()` returns every tag with that name, as a list, and `len()` counts a list's items. Then we count the parties. The given cell's `for` loop runs its indented lines once for each member, and `.append()` adds that member's party to the end of the list `parties`. `Counter` tallies a list: how many times each value appears. Read what it prints before you trust it. See "Working with Real XML".

```
# members: every "member" tag in soup, with soup.find_all()
members = soup.find_all("member")
```

```
# Print how many members there are, with len()
print(len(members))
```

```
441
```

```
# Given: every member's party, in a list
parties = []
for m in members:
    parties.append(m.find("party").text)
```

```
# Count the parties: Counter(parties)
Counter(parties)
```

```
Counter({'R': 221, 'D': 217, '': 2, 'I': 1})
```

⏱️ Microsprint (5 minutes, alone or in pairs)

Do: the House has 435 seats, and we counted 441 members. Run the given cell, which lists every member's `<district>`. Then count how many say `"Delegate"` and how many say `"Resident Commissioner"`, with a list's `.count()`. **Share:** the two numbers, and who you think these members are.

```
# Given: every member's district, in a list
districts = []
for m in members:
    districts.append(m.find("district").text)
```

```
# Microsprint: print districts.count("Delegate"), and for "Resident Commissioner"
print(districts.count("Delegate"))
print(districts.count("Resident Commissioner"))
```

```
5
1
```

💬 Discuss

- The saved roster has 441 entries. How many of them are members who vote, and what would a paper that reported "441 representatives" have gotten wrong?
- `soup.find_all("Member")`, with a capital M, returns an empty list, with no error. In your RSS feed the date tag is `<pubDate>`. What will you check?
- `Counter(parties)` shows two empty strings, `''`. A parsing bug, or something in the file? How would you find out?

Step 5 — Read the odd entries: a vacant seat (8 minutes)

The two empty parties are seats with no member: **vacancies**. Their `<member>` tags are still in the file. The given cell's `if` keeps only the members whose party is the empty string, `""`; we read the first by hand. The Clerk's guide says `<lastname>` "will not be empty": check. A `<footnote>` says why the seat is vacant. Footnotes name people, so we print one and count the rest. See the end of "Working with Real XML".

```
# Given: the members whose party is the empty string, ""
empty_seats = []
for m in members:
    if m.find("party").text == "":
        empty_seats.append(m)
print(len(empty_seats))
```

```
2
```

```
# vacant: empty_seats[0]. Print the text of its "statedistrict"
vacant = empty_seats[0]
print(vacant.find("statedistrict").text)
```

```
FL20
```

```
# Look at its "lastname" tag itself, without .text
vacant.find("lastname")
```

```
<lastname/>
```

```
# Print the text of its "district" tag, then of its "footnote" tag
print(vacant.find("district").text)
print(vacant.find("footnote").text)
```

```
20th
Vacancy due to the resignation of Sheila Cherfilus-McCormick, April 21, 2026.
```

⏱️ Microsprint (3 minutes, alone or in pairs)

Do: count every `<footnote>` tag in the roster, with `soup.find_all()` and `len()`. Count them; don't print them all. **Share:** the number, and a guess at why there are more footnotes than vacant seats.

```
# Microsprint: print how many "footnote" tags soup has
print(len(soup.find_all("footnote")))
```

```
13
```

💬 Discuss

- Which of the vacant seat's tags were empty, and which still held something? Could you still put the seat on a map?
- The guide says `<lastname>` "will not be empty", and here it is. When a file and its documentation disagree, which do you report, and how?
- Would you drop the two vacancies from a count of the House, or keep them and say why?

Part 2 — Wednesday: from a tree to a table, and JSON's two shapes

On Monday we read one member by hand and counted the parties. Today each member becomes one row of a table, the chapter's pipeline; we use the table, and see what `.text` would have done to five names. Then JSON: lists and dictionaries, written as text.

Step 6 — Set up again (4 minutes)

Part 2 starts fresh, so a student who missed Monday can start here. Put your email address in the User-Agent, and run the given cell. It loads the course's saved roster, published November 3, 2026, and finds its members as Monday did; it sends nothing to the Clerk. `json` is Python's module for JSON text, and pandas, imported as `pd`, makes tables.

```
# Given: Part 2's setup. Put your email address in HEADERS, then run it.
# It loads the roster the key's run saved on 2026-11-11 (published November 3, 2026)
import json

import pandas as pd
import requests
```

```
from bs4 import BeautifulSoup

HEADERS = {"User-Agent": "WebDataScience/1.0 (your-email@colorado.edu)"}
data_base =
 ↪  "https://raw.githubusercontent.com/cuinfoscience/INFO4617-Fall2026/main/labs/week-04/data/"
saved_response = requests.get(data_base + "MemberData-2026-11-03.xml", headers=HEADERS)
soup = BeautifulSoup(saved_response.content, "xml")
members = soup.find_all("member")
print(soup.find("MemberData")["publish-date"], len(members))
```

```
November 3, 2026 441
```

💬 Discuss

- A new Congress begins on January 3 of each odd-numbered year. What will the live roster say then that our saved copy can't?

Step 7 — One record by hand, then every member (14 minutes)

A **record** is one row's worth of data, kept as a dictionary: a key for each column, and its value. Build one for the first member, one key at a time: `record["party"] = …` adds the key `"party"` and its value. Then the given cell does the same for every member, collects the records in a list, and makes a table with `pd.DataFrame()`. Find your lines in it. This is the chapter's pipeline, from "From XML to DataFrame".

```
# first: members[0]. record: an empty dictionary, {}
first = members[0]
record = {}
```

```
# Add "last_name" (the text of first's "lastname") and "party" (the text of its "party")
record["last_name"] = first.find("lastname").text
record["party"] = first.find("party").text
```

```
# Add "state" (the "state" tag's "postal-code") and "district" (the text of "district")
record["state"] = first.find("state")["postal-code"]
record["district"] = first.find("district").text
```

```
# Print record
print(record)
```

```
{'last_name': 'Begich', 'party': 'R', 'state': 'AK', 'district': 'At Large'}
```

```
# Given: a record like ours for every member, with two more keys, collected in a list.
# Then the list becomes a table: one row for each record, one column for each key
records = []
for m in members:
    row = {}
    row["first_name"] = m.find("firstname").text
    row["last_name"] = m.find("lastname").text
    row["party"] = m.find("party").text
    row["caucus"] = m.find("caucus").text
    row["state"] = m.find("state")["postal-code"]
    row["district"] = m.find("district").text
    records.append(row)
df = pd.DataFrame(records)
```

```
# Look at the table's first rows, with df.head()
df.head()
```

```
  first_name last_name party caucus state  district
0   Nicholas    Begich     R      R    AK  At Large
1      Barry     Moore     R      R    AL       1st
2    Shomari   Figures     D      D    AL       2nd
3       Mike    Rogers     R      R    AL       3rd
4     Robert  Aderholt     R      R    AL       4th
```

```
# Given: save the table as Wednesday's checkpoint, beside this notebook
df.to_csv("lab04_roster.csv", index=False)
```

```
# Given, only if the cells above stopped: remove the "#" from the next two lines, and
# run this cell. It loads the table the key's run saved on 2026-11-11, and makes records
# again. keep_default_na=False keeps the vacancies' empty strings as empty strings
# df = pd.read_csv(data_base + "lab04_roster.csv", keep_default_na=False)
# records = df.to_dict("records")
```

💬 **Discuss**

- The chapter guards each field with `if m.find(...) else None`, in case a tag is missing. The vacant seats have every one of these tags, some of them empty. Would the guards have caught them? What did?
- In week 1, one call, `pd.DataFrame(data["items"])`, made a table from the pageviews JSON. Why did the roster need a loop?
- Every value in our table is text, the district too. Which column would you change before analyzing it, and into what?

Step 8 — Use the table (14 minutes)

`.value_counts()` counts each value in a column. Double brackets, `df[["party", "caucus"]]`, pick two columns, and `.value_counts()` then counts the pairs. A comparison, `df["state"] == "CO"`, is True or False for each row, and `df[...]` keeps the rows where it's True; `!=` means "is not". The given cell counts each state's members of each party with `pd.crosstab()`, and the states with members of both parties: `&` means "and". `.loc["CO"]` picks the row labeled `CO`.

```
# Count the values in df["party"], with .value_counts()
df["party"].value_counts()
```

```
party
R    221
D    217
       2
I      1
Name: count, dtype: int64
```

```
# Count the pairs of party and caucus: df[["party", "caucus"]].value_counts()
df[["party", "caucus"]].value_counts()
```

```
party  caucus
R      R         221
D      D         217
                   2
I      R           1
Name: count, dtype: int64
```

```
# Colorado's members: the rows where df["state"] == "CO"
df[df["state"] == "CO"]
```

```
   first_name  last_name party caucus state district
74      Diana    DeGette     D      D    CO      1st
75        Joe     Neguse     D      D    CO      2nd
76       Jeff       Hurd     R      R    CO      3rd
77     Lauren    Boebert     R      R    CO      4th
78       Jeff      Crank     R      R    CO      5th
79      Jason       Crow     D      D    CO      6th
80   Brittany  Pettersen     D      D    CO      7th
81       Gabe      Evans     R      R    CO      8th
```

```
# serving: the rows whose "party" != "" (not empty). Print how many
serving = df[df["party"] != ""]
print(len(serving))
```

```
439
```

```
# Given: for each state, its members of each party, and its seats. Then the states with
# two seats or more, and of those, the ones with at least one Democrat and one Republican
table = pd.crosstab(serving["state"], serving["party"])
table["seats"] = table["D"] + table["I"] + table["R"]
several = table[table["seats"] >= 2]
split = several[(several["D"] > 0) & (several["R"] > 0)]
print(len(split), "of", len(several),
      "states with two seats or more have members of both parties")
```

```
29 of 44 states with two seats or more have members of both parties
```

⏱️ Microsprint (4 minutes, alone or in pairs)

Do: print Colorado's row of `table`, with `table.loc["CO"]`, then another state's. **Share:** each state's split, and whether a state with one seat could ever be counted as split.

```
# Microsprint: table.loc["CO"], Colorado's row. Then try another state's postal code
table.loc["CO"]
```

```
party
D        4
I        0
R        4
seats    8
Name: CO, dtype: int64
```

💬 Discuss

- One member's party is `I` and his caucus is `R`. Name a research question for which he counts as a Republican, and one for which he doesn't.
- `df.dropna()` drops no rows here: the vacancies are empty strings, not missing values. What would a study that "dropped the missing values" have counted?
- Why does the given cell count split states out of those with two seats or more, and not out of every state and territory?

Step 9 — What `.text` would have done (6 minutes)

On Monday, `requests` guessed ISO-8859-1, an older alphabet, while the file declared UTF-8, which writes "á" as two bytes; ISO-8859-1 reads each byte as a letter of its own. `.text` decodes the bytes

with that guess. The given cell reads the saved bytes the same wrong way, parses them, and puts each member's last name, as guessed, in a new column. Then keep the rows whose two last names differ. The chapter's "Common Issues to Debug" names this problem.

```
# Given: the saved bytes read with requests' guess, ISO-8859-1, as .text read the Clerk's.
# Each member's last name, as guessed, goes in a new column, as_guessed
guessed_text = saved_response.content.decode("ISO-8859-1")
guessed_soup = BeautifulSoup(guessed_text, "xml")
as_guessed = []
for m in guessed_soup.find_all("member"):
    as_guessed.append(m.find("lastname").text)
df["as_guessed"] = as_guessed
```

```
# The rows where df["last_name"] != df["as_guessed"]
df[df["last_name"] != df["as_guessed"]]
```

```
    first_name  last_name party caucus state               district  \
59       Linda    Sánchez     D      D    CA                   38th
65     Nanette   Barragán     D      D    CA                   44th
143      Jesús     García     D      D    IL                    4th
282      Nydia  Velázquez     D      D    NY                    7th
345      Pablo  Hernández     D      D    PR  Resident Commissioner

     as_guessed
59     SÃ¡nchez
65    BarragÃ¡n
143     GarcÃa
282  VelÃ¡zquez
345  HernÃ¡ndez
```

💬 Discuss

- The wrong alphabet parsed with no error and no warning. How would you have noticed, and which of Monday's prints was the clue?
- Five names out of 441 is about one percent. For which analysis would it matter, and for which wouldn't it?

Step 10 — JSON is lists and dictionaries (12 minutes)

JSON is text that writes lists and dictionaries: a JSON *array*, `[…]`, is a Python list, and a JSON *object*, `{…}`, is a dictionary. `records` is already a list of dictionaries. `json.dumps()` writes Python as JSON text, and `json.loads()` reads JSON text back into Python; the `s` is for *string*. A table can also come from a dictionary of lists, matched by position. See "Lists and Dictionaries" and "Parsing JSON Strings".

```
# first_two: a list of records[0] and records[1]
first_two = [records[0], records[1]]
```

```
# Print the second record's "last_name": first_two[1], then the key
print(first_two[1]["last_name"])
```

```
Moore
```

```
# Given: first_two written as JSON text. Compare it with how Python prints the same list
print(json.dumps(first_two, indent=2))
```

```
[
  {
    "first_name": "Nicholas",
    "last_name": "Begich",
```

```
    "party": "R",
    "caucus": "R",
    "state": "AK",
    "district": "At Large"
  },
  {
    "first_name": "Barry",
    "last_name": "Moore",
    "party": "R",
    "caucus": "R",
    "state": "AL",
    "district": "1st"
  }
]
```

```
# colorado: json.loads() the string '{"state": "CO", "seats": 8}'
colorado = json.loads('{"state": "CO", "seats": 8}')
```

```
# Print type(colorado), and its "seats"
print(type(colorado), colorado["seats"])
```

```
<class 'dict'> 8
```

```
# Given: a table from a dictionary of lists. Item 0 of each list makes row 0, and so on
seats_table = pd.DataFrame({"state": ["CO", "WY", "CA"], "seats": [8, 1, 52]})
seats_table
```

```
  state  seats
0    CO      8
1    WY      1
2    CA     52
```

⏱️ Microsprint (5 minutes, alone or in pairs)

Do: each cell below gives `json.loads()` one broken string, of the kinds the chapter's "Common Issues to Debug" lists: single quotes, a comma before the closing brace, and an HTML page where JSON should be. Each one stops with an error: that's the point. Type each as its prompt spells it, run it, and read the last line of its error. **Share:** the three messages, and which one you'd expect from a live API that refused you.

```
# Microsprint: json.loads("{'state': 'CO'}"), single quotes inside
json.loads("{'state': 'CO'}")
```

```
---------------------------------------------------------------------------
JSONDecodeError                           Traceback (most recent call last)
Cell In[52], line 2
      1 # Microsprint: json.loads("{'state': 'CO'}"), single quotes inside
----> 2 json.loads("{'state': 'CO'}")

File
 ↪  ~/.local/share/uv/python/cpython-3.14.6-linux-x86_64-gnu/lib/python3.14/json/__init__.py:352,
 ↪  in loads(s, cls, object_hook, parse_float, parse_int, parse_constant, object_pairs_hook,
 ↪  **kw)
    347     s = s.decode(detect_encoding(s), 'surrogatepass')
    349 if (cls is None and object_hook is None and
    350         parse_int is None and parse_float is None and
    351         parse_constant is None and object_pairs_hook is None and not kw):
--> 352     return _default_decoder.decode(s)
```

```
    353 if cls is None:
    354     cls = JSONDecoder

File ~/.local/share/uv/python/cpython-3.14.6-linux-x86_64-gnu/lib/python3.14/json/decoder.py:345,
 ↪  in JSONDecoder.decode(self, s, _w)
    340 def decode(self, s, _w=WHITESPACE.match):
    341     """Return the Python representation of ``s`` (a ``str`` instance
    342     containing a JSON document).
    343
    344     """
--> 345     obj, end = self.raw_decode(s, idx=_w(s, 0).end())
    346     end = _w(s, end).end()
    347     if end != len(s):

File ~/.local/share/uv/python/cpython-3.14.6-linux-x86_64-gnu/lib/python3.14/json/decoder.py:361,
 ↪  in JSONDecoder.raw_decode(self, s, idx)
    352 """Decode a JSON document from ``s`` (a ``str`` beginning with
    353 a JSON document) and return a 2-tuple of the Python
    354 representation and the index in ``s`` where the document ended.
   (...)    358
    359 """
    360 try:
--> 361     obj, end = self.scan_once(s, idx)
    362 except StopIteration as err:
    363     raise JSONDecodeError("Expecting value", s, err.value) from None

JSONDecodeError: Expecting property name enclosed in double quotes: line 1 column 2 (char 1)
```

```
# Microsprint: json.loads('{"state": "CO",}'), a comma before the closing brace
json.loads('{"state": "CO",}')
```

```
---------------------------------------------------------------------------
JSONDecodeError                           Traceback (most recent call last)
Cell In[53], line 2
      1 # Microsprint: json.loads('{"state": "CO",}'), a comma before the closing brace
----> 2 json.loads('{"state": "CO",}')

File
 ↪  ~/.local/share/uv/python/cpython-3.14.6-linux-x86_64-gnu/lib/python3.14/json/__init__.py:352,
 ↪  in loads(s, cls, object_hook, parse_float, parse_int, parse_constant, object_pairs_hook,
 ↪  **kw)
    347     s = s.decode(detect_encoding(s), 'surrogatepass')
    349 if (cls is None and object_hook is None and
    350         parse_int is None and parse_float is None and
    351         parse_constant is None and object_pairs_hook is None and not kw):
--> 352     return _default_decoder.decode(s)
    353 if cls is None:
    354     cls = JSONDecoder

File ~/.local/share/uv/python/cpython-3.14.6-linux-x86_64-gnu/lib/python3.14/json/decoder.py:345,
 ↪  in JSONDecoder.decode(self, s, _w)
    340 def decode(self, s, _w=WHITESPACE.match):
    341     """Return the Python representation of ``s`` (a ``str`` instance
    342     containing a JSON document).
    343
    344     """
--> 345     obj, end = self.raw_decode(s, idx=_w(s, 0).end())
```

```
    346     end = _w(s, end).end()
    347     if end != len(s):

File ~/.local/share/uv/python/cpython-3.14.6-linux-x86_64-gnu/lib/python3.14/json/decoder.py:361,
 ↪  in JSONDecoder.raw_decode(self, s, idx)
    352 """Decode a JSON document from ``s`` (a ``str`` beginning with
    353 a JSON document) and return a 2-tuple of the Python
    354 representation and the index in ``s`` where the document ended.
   (...)    358
    359 """
    360 try:
--> 361     obj, end = self.scan_once(s, idx)
    362 except StopIteration as err:
    363     raise JSONDecodeError("Expecting value", s, err.value) from None

JSONDecodeError: Illegal trailing comma before end of object: line 1 column 15 (char 14)
```

```
# Microsprint: json.loads("<html><body>Too Many Requests</body></html>")
json.loads("<html><body>Too Many Requests</body></html>")
```

```
---------------------------------------------------------------------------
JSONDecodeError                           Traceback (most recent call last)
Cell In[54], line 2
      1 # Microsprint: json.loads("<html><body>Too Many Requests</body></html>")
----> 2 json.loads("<html><body>Too Many Requests</body></html>")

File
 ↪  ~/.local/share/uv/python/cpython-3.14.6-linux-x86_64-gnu/lib/python3.14/json/__init__.py:352,
 ↪  in loads(s, cls, object_hook, parse_float, parse_int, parse_constant, object_pairs_hook,
 ↪  **kw)
    347     s = s.decode(detect_encoding(s), 'surrogatepass')
    349 if (cls is None and object_hook is None and
    350         parse_int is None and parse_float is None and
    351         parse_constant is None and object_pairs_hook is None and not kw):
--> 352     return _default_decoder.decode(s)
    353 if cls is None:
    354     cls = JSONDecoder

File ~/.local/share/uv/python/cpython-3.14.6-linux-x86_64-gnu/lib/python3.14/json/decoder.py:345,
 ↪  in JSONDecoder.decode(self, s, _w)
    340 def decode(self, s, _w=WHITESPACE.match):
    341     """Return the Python representation of ``s`` (a ``str`` instance
    342     containing a JSON document).
    343
    344     """
--> 345     obj, end = self.raw_decode(s, idx=_w(s, 0).end())
    346     end = _w(s, end).end()
    347     if end != len(s):

File ~/.local/share/uv/python/cpython-3.14.6-linux-x86_64-gnu/lib/python3.14/json/decoder.py:363,
 ↪  in JSONDecoder.raw_decode(self, s, idx)
    361     obj, end = self.scan_once(s, idx)
    362 except StopIteration as err:
--> 363     raise JSONDecodeError("Expecting value", s, err.value) from None
    364 return obj, end

JSONDecodeError: Expecting value: line 1 column 1 (char 0)
```

💬 **Discuss**

- In the key's run, on Python 3.14, the comma's message begins ”Illegal trailing comma”; Python 3.12 says ”Expecting property name”. If yours differs from your neighbor's, what does that tell you?
- `colorado["seats"]` came back as a number, but every value in our roster table was text. Which format decided that?
- The House publishes its roster as XML; on Friday, NASA sends its weather as JSON. Using the chapter's ”When to Use Which”, why might each have chosen its format?

Part 3 — Friday: one key at a time, and is it the right place?

NASA's POWER API answers with daily weather for any point on Earth, as JSON, with no key and no account. We ask it for Boulder's 2025, read one refusal on purpose, open the answer one key at a time, make a table and a plot, and check that the numbers describe the place we meant.

Step 11 — Set up (3 minutes)

Put your email address in the User-Agent, and run the given cell. Today needs nothing from Monday or Wednesday. An **API** is a web address that answers a program with data instead of a page. `power_url` is the address of POWER's Daily API for one point. Its documentation lists each setting a request needs; the settings go in a dictionary, as in the chapter.

```
# Given: Part 3's setup. Put your email address in HEADERS, then run it
from pathlib import Path

import matplotlib.pyplot as plt
import pandas as pd
import requests

HEADERS = {"User-Agent": "WebDataScience/1.0 (your-email@colorado.edu)"}
power_url = "https://power.larc.nasa.gov/api/temporal/daily/point"
data_base =
 ↪  "https://raw.githubusercontent.com/cuinfoscience/INFO4617-Fall2026/main/labs/week-04/data/"
```

💬 **Discuss**

- POWER's `robots.txt` disallows `/api/` for every crawler, as Wikipedia's disallows `/w/` (week 2), and a second block names the crawlers of AI companies. A crawler follows links from page to page. Are we one?
- Would it matter if an AI assistant sent these requests for you?

Step 12 — Ask POWER, look, and read a refusal (8 minutes)

The given `weather_params` asks for Boulder's daily high (`T2M_MAX`), low (`T2M_MIN`) and precipitation (`PRECTOTCORR`) in 2025, a finished year, so everyone gets the same answer. `"community": "AG"` picks POWER's agriculture units. Look before you read: POWER answers a mistake with status 422 and a message. `.copy()` makes a second dictionary, and `.pop()` takes one key out of it, so we can break a request on purpose.

```
# Given: the request's settings: Boulder's latitude and longitude, and the whole of 2025
weather_params = {"parameters": "T2M_MAX,T2M_MIN,PRECTOTCORR", "community": "AG",
                  "latitude": 40.01, "longitude": -105.27,
                  "start": "20250101", "end": "20251231"}
```

```
# weather_response: requests.get() power_url, with params=weather_params, headers=HEADERS
weather_response = requests.get(power_url, params=weather_params, headers=HEADERS)
```

```
# Print its status code, and the first 200 characters of its .text
print(weather_response.status_code, weather_response.text[:200])
```

```
200 {"type":"Feature","geometry":{"type":"Point","coordinates":[-105.27,40.01,1801.15]},"propert
↪  ies":{"parameter":{"T2M_MAX":{"20250101":4.5,"20250102":7.86,"20250103":9.47,"20250104":3.82
↪  ,"20250105":0.13
```

```
# Print weather_response.url, the whole address requests sent. Open it in a browser tab
print(weather_response.url)
```

```
https://power.larc.nasa.gov/api/temporal/daily/point?parameters=T2M_MAX%2CT2M_MIN%2CPRECTOTCORR&
↪  community=AG&latitude=40.01&longitude=-105.27&start=20250101&end=20251231
```

```
# Given, only if the request above stopped: remove the "#" from the next line, and run
# this cell. It loads POWER's answer as the key's run saved it on 2026-11-11
# weather_response = requests.get(data_base + "lab04_boulder_2025.json", headers=HEADERS)
```

```
# Given: save POWER's answer exactly as it came, beside this notebook
Path("lab04_boulder_2025.json").write_bytes(weather_response.content)
```

```
18560
```

```
# broken_params: weather_params.copy(). Then broken_params.pop("community")
broken_params = weather_params.copy()
broken_params.pop("community")
```

```
'AG'
```

```
# broken_response: send broken_params; print its status code and first 200 characters
broken_response = requests.get(power_url, params=broken_params, headers=HEADERS)
print(broken_response.status_code, broken_response.text[:200])
```

```
422 {"detail":[{"type":"missing","loc":["query","community"],"msg":"Field
↪  required","input":null}]}
```

```
# broken_data: broken_response.json(); then broken_data["geometry"], as if it had worked
broken_data = broken_response.json()
broken_data["geometry"]
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
Cell In[64], line 3
      1 # broken_data: broken_response.json(); then broken_data["geometry"], as if it had worked
      2 broken_data = broken_response.json()
----> 3 broken_data["geometry"]

KeyError: 'geometry'
```

💬 Discuss

- POWER said in words what was wrong. Without our print, which line would have stopped, and with what error?
- POWER's documentation lists one more failure, 429, Too Many Requests, and thirty of us share one campus address. What would our print show then? Does the chapter's Open-Meteo code guard against it?
- Typed without `.copy()`, `broken_params = weather_params` gives the same dictionary a second name, and `.pop()` takes `"community"` out of both, with no error. Which later cell would be the first to fail?

Step 13 — Peel the onion (6 minutes)

`.json()` turns the answer's text into Python: a dictionary with dictionaries inside. Don't guess the path to the numbers. **Peel the onion**, the chapter's name for opening one layer at a time: print the keys, pick one key and print its keys, and repeat until you see numbers. See "Handling Deeply Nested JSON".

```
# weather_data: weather_response.json(). Print its type()
weather_data = weather_response.json()
print(type(weather_data))
```

```
<class 'dict'>
```

```
# Look at its keys, weather_data.keys()
weather_data.keys()
```

```
dict_keys(['type', 'geometry', 'properties', 'header', 'messages', 'parameters', 'times'])
```

```
# Look at weather_data["geometry"]
weather_data["geometry"]
```

```
{'type': 'Point', 'coordinates': [-105.27, 40.01, 1801.15]}
```

```
# Look at the keys of weather_data["properties"]
weather_data["properties"].keys()
```

```
dict_keys(['parameter'])
```

```
# One layer deeper: the keys of weather_data["properties"]["parameter"]
weather_data["properties"]["parameter"].keys()
```

```
dict_keys(['T2M_MAX', 'T2M_MIN', 'PRECTOTCORR'])
```

```
# One day's high: weather_data["properties"]["parameter"]["T2M_MAX"]["20250101"]
weather_data["properties"]["parameter"]["T2M_MAX"]["20250101"]
```

```
4.5
```

```
# Look at weather_data["header"]
weather_data["header"]
```

```
{'title': 'NASA/POWER Source Native Resolution Daily Data',
 'api': {'version': 'v2.10.3', 'name': 'POWER Daily API'},
 'sources': ['MERRA2', 'POWER'],
 'fill_value': -999.0,
 'time_standard': 'LST',
 'start': '20250101',
 'end': '20251231'}
```

💬 Discuss

- `"coordinates"` came back as three numbers. Which is the latitude, how would you know, and what is the third?
- Here each day's value sits under its date, `"20250101"`. In the chapter's forecast the dates are a list of their own, beside a list of values. Which is easier to get wrong, and how?

Step 14 — Into a table, and the days that are missing (9 minutes)

`weather_data["properties"]["parameter"]` holds three dictionaries, each mapping a date to a number; `pd.DataFrame()` lines them up, one row per date. POWER writes a **fill value**, −999, where a value is missing, and the header names it. `.replace()` swaps it for `float("nan")`, pandas' mark for "no value",

which averages skip. A comparison is True or False for each day, and `.sum()` counts the Trues. The given plot uses `plt.subplots()`, the course's one way to plot.

```
# weather: pd.DataFrame() of weather_data["properties"]["parameter"]; then weather.head()
weather = pd.DataFrame(weather_data["properties"]["parameter"])
weather.head()
```

```
          T2M_MAX  T2M_MIN  PRECTOTCORR
20250101     4.50    -7.35         0.38
20250102     7.86    -4.40         2.75
20250103     9.47    -2.41         0.08
20250104     3.82    -6.12         0.91
20250105     0.13    -7.76         0.71
```

```
# Given: turn the row labels into dates, and give the columns short names
weather.index = pd.to_datetime(weather.index, format="%Y%m%d")
weather = weather.rename(columns={"T2M_MAX": "high", "T2M_MIN": "low",
                                  "PRECTOTCORR": "precip"})
weather.head()
```

```
            high   low  precip
2025-01-01  4.50 -7.35    0.38
2025-01-02  7.86 -4.40    2.75
2025-01-03  9.47 -2.41    0.08
2025-01-04  3.82 -6.12    0.91
2025-01-05  0.13 -7.76    0.71
```

```
# fill: weather_data["header"]["fill_value"]. Print it
fill = weather_data["header"]["fill_value"]
print(fill)
```

```
-999.0
```

```
# Replace fill with float("nan") in weather; then count what's missing: .isna().sum()
weather = weather.replace(fill, float("nan"))
weather.isna().sum()
```

```
high      0
low       0
precip    0
dtype: int64
```

```
# Print the mean high, weather["high"].mean()
print(weather["high"].mean())
```

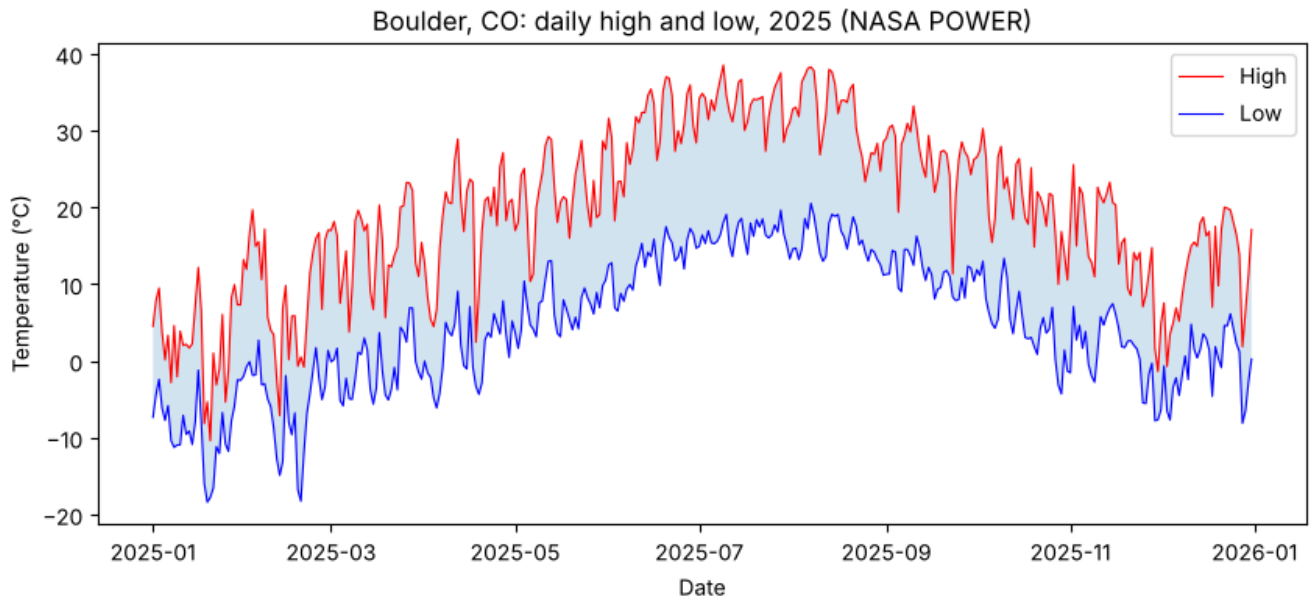
```
19.74819178082192
```

```
# Print how many nights the low fell below 0 °C: (weather["low"] < 0).sum()
print((weather["low"] < 0).sum())
```

```
115
```

```
# Given: the year's highs and lows, as the chapter plots its forecast
fig, ax = plt.subplots(figsize=(10, 4))
ax.plot(weather.index, weather["high"], label="High", color="red", linewidth=0.8)
ax.plot(weather.index, weather["low"], label="Low", color="blue", linewidth=0.8)
ax.fill_between(weather.index, weather["low"], weather["high"], alpha=0.2)
ax.set_title("Boulder, CO: daily high and low, 2025 (NASA POWER)")
ax.set_xlabel("Date")
ax.set_ylabel("Temperature (°C)")
```

```
ax.legend()
plt.show()
```



⏱️ Microsprint (5 minutes, alone or in pairs)

Do: ask POWER for Boulder's most recent ten days. Make `recent_params`, a copy of `weather_params` whose `"start"` is ten days ago and whose `"end"` is today, both written as `YYYYMMDD`. Then run the given cell. **Share:** how many of the days came back as the fill value, and the mean high before and after `.replace()`.

```
# Microsprint: recent_params, a .copy() of weather_params, with your own "start" and "end"
recent_params = weather_params.copy()
recent_params["start"] = "20261101"
recent_params["end"] = "20261111"
```

```
# Given: send recent_params. Print the last days, and the mean high before and after
recent_response = requests.get(power_url, params=recent_params, headers=HEADERS)
print(recent_response.status_code)
recent_data = recent_response.json()
recent = pd.DataFrame(recent_data["properties"]["parameter"])
print(recent.tail(4))
print("Mean high as sent:", round(recent["T2M_MAX"].mean(), 1))
recent = recent.replace(recent_data["header"]["fill_value"], float("nan"))
print("Mean high after .replace():", round(recent["T2M_MAX"].mean(), 1))
```

```
200
          T2M_MAX  T2M_MIN  PRECTOTCORR
20261108    19.26     4.34          0.0
20261109  -999.00  -999.00       -999.0
20261110  -999.00  -999.00       -999.0
20261111  -999.00  -999.00       -999.0
Mean high as sent: -261.7
Mean high after .replace(): 14.7
```

💬 Discuss

- For 2025, `.isna().sum()` printed 0 in every column. Is that the answer you wanted? Without the `.replace()` line, what would the microsprint's mean have looked like?
- Typed as `weather_data["parameters"]`, one letter more, the `pd.DataFrame()` line runs, and the given cell after it stops with a message about the date format. Which line was wrong, and how would you find it?
- POWER's numbers come from a model, a computer simulation of the weather corrected with measurements, for a square of its grid about 50 km across. Would you write "Boulder had this many freezing nights in 2025"? What would you write instead?

Step 15 — Is it the place you meant? (7 minutes)

The given `get_daily()` is a **function**: lines with a name, run each time it's called. `def` names its inputs, and `return` hands back its table. It repeats Steps 12 to 14 for any point in 2025: find your lines in it. It prints POWER's status, then the point POWER used: longitude, latitude, and the **grid elevation**, the height of the model's square. Call it for Leadville, at about 3,100 m, then for Boulder with the minus sign left off, on purpose.

```
# Given: Steps 12 to 14 for any point, in 2025. It prints the status and the start of the
# answer, then POWER's point: longitude, latitude, and the grid's elevation in meters
def get_daily(latitude, longitude):
    place_params = {"parameters": "T2M_MAX,T2M_MIN,PRECTOTCORR", "community": "AG",
                    "latitude": latitude, "longitude": longitude,
                    "start": "20250101", "end": "20251231"}
    place_response = requests.get(power_url, params=place_params, headers=HEADERS)
    print(place_response.status_code, place_response.text[:100])
    place_data = place_response.json()
    print("POWER's point:", place_data["geometry"]["coordinates"])
    place = pd.DataFrame(place_data["properties"]["parameter"])
    place.index = pd.to_datetime(place.index, format="%Y%m%d")
    place = place.rename(columns={"T2M_MAX": "high", "T2M_MIN": "low",
                                  "PRECTOTCORR": "precip"})
    place = place.replace(place_data["header"]["fill_value"], float("nan"))
    return place
```

```
# leadville: get_daily(39.25, -106.29)
leadville = get_daily(39.25, -106.29)
```

```
200 {"type":"Feature","geometry":{"type":"Point","coordinates":[-106.29,39.25,3233.91]},"propert
 ↪  ies":{"p
POWER's point: [-106.29, 39.25, 3233.91]
```

```
# Print Leadville's mean high, and its nights below 0 °C, as Step 14 did for Boulder
print(leadville["high"].mean(), (leadville["low"] < 0).sum())
```

```
8.12868493150685 252
```

```
# no_minus: get_daily(40.01, 105.27), Boulder with the minus sign left off, on purpose
no_minus = get_daily(40.01, 105.27)
```

```
200 {"type":"Feature","geometry":{"type":"Point","coordinates":[105.27,40.01,1428.85]},"properti
 ↪  es":{"pa
POWER's point: [105.27, 40.01, 1428.85]
```

```
# Given: one row for each place, from a dictionary of lists, as in Step 10
highs = [weather["high"].mean(), leadville["high"].mean(), no_minus["high"].mean()]
nights = [(weather["low"] < 0).sum(), (leadville["low"] < 0).sum(),
          (no_minus["low"] < 0).sum()]
places = pd.DataFrame({"place": ["Boulder", "Leadville", "40.01, 105.27 (no minus)"],
```

```
                      "mean_high_c": highs, "nights_below_0": nights})
places.round(1)
```

```
                     place  mean_high_c  nights_below_0
0                  Boulder         19.7             115
1                Leadville          8.1             252
2  40.01, 105.27 (no minus)         14.8             171
```

```
# Given: save the table as Friday's checkpoint, beside this notebook
places.to_csv("lab04_places.csv", index=False)
```

```
# Given, only if the cells above stopped: remove the "#" from the next line, and run this
# cell. It loads the table the key's run saved on 2026-11-11
# places = pd.read_csv(data_base + "lab04_places.csv")
```

⏱️ Microsprint (5 minutes, alone or in pairs)

Do: call `get_daily()` for a place you know, such as your hometown; its Wikipedia article gives its coordinates, and west and south are negative. Before you read the weather, check POWER's point and its elevation against what you know. The key uses Grand Junction, Colorado, at 39.06, -108.55. **Share:** your place, POWER's point and elevation, and its mean high and nights below 0 °C, beside the table.

```
# Microsprint: mine, get_daily() for your place; print its mean high and nights below 0 °C
mine = get_daily(39.06, -108.55)
print(mine["high"].mean(), (mine["low"] < 0).sum())
```

```
200 {"type":"Feature","geometry":{"type":"Point","coordinates":[-108.55,39.06,1904.16]},"propert
↪  ies":{"p
POWER's point: [-108.55, 39.06, 1904.16]
18.85591780821918 122
```

💬 Discuss

- POWER answered 40.01, 105.27 without a word, with weather that looks like weather. What in its answer told you it was not Boulder?
- POWER's documentation allows latitudes from −90 to 90 and longitudes from −180 to 180. If a place's two numbers were swapped, when would POWER refuse, and when would it answer for somewhere else?
- POWER's documentation warns that an application that keeps asking for the same location may be blocked. If each of us asked for 50 places, how many requests would this room send?

Step 16 — What did we learn? (2 minutes)

In a few sentences, as comments:

- In the saved roster (published November 3, 2026), how many of the 441 entries are members serving who vote? What in the XML told you about the rest?
- Which mistakes this week stopped with an error, and which gave a wrong answer with no error? Name one of each.
- How many nights below 0 °C did Boulder's grid square have in 2025, and how did Leadville and your place compare? What does the grid elevation say about what these numbers describe?
- Besides the data, what did each source tell you about itself (`publish-date`, units, `fill_value`)? Which would you save beside your table?

Recommended Exercises ahead (in the chapter's companion notebook, due Sunday):

- Step 1, `HEADERS` and an RSS feed you've checked against its `robots.txt`: the setup steps' `HEADERS`, and the `robots.txt` questions after Steps 1 and 11. `handouts/week-04/rss-feeds.md` lists starter feeds.
- Step 2, fetch the feed and look at the raw XML: Step 1's status code, `Content-Type` and first bytes, and Step 12's first characters of `.text`.
- Step 3, parse it and count the items: Step 2's `"xml"` parser, and Step 4's `find_all()` and `len()`. In a feed the repeating tag is `<item>`, and tag names are case-sensitive (`<pubDate>`).
- Step 4, the items into a DataFrame: Step 7's record by hand, and its given loop.
- Step 5, fetch the forecast and map its keys: Steps 12 and 13, one key at a time.
- Step 6, a DataFrame from the forecast's parallel lists, and a plot: Step 10's table from a dictionary of lists, and Step 14's given plot.
- Step 7, interpret: the Discuss questions after Steps 7 and 13.

```
# Your answers here, as comments
# 1. 439 of the 441 entries are people, and 433 of them vote: FL20 and TX23 are vacant seats,
 ↪  whose
#    <party> and <lastname> are empty and whose <footnote> says the member resigned, and 6
 ↪  members are
#    non-voting (5 with the district "Delegate", 1 "Resident Commissioner"). Counter(parties)
 ↪  alone said
#    R 221, D 217, '' 2, I 1: the '' looked like a parsing bug, and the I hid that his caucus is
 ↪  R.
# 2. With an error: find("lastName") gave None, and .text on it stopped with AttributeError;
 ↪  POWER's 422
#    became KeyError: 'geometry'; the three broken strings each stopped with JSONDecodeError.
 ↪  With no
#    error: state_tag.text gave "Alaska" for the postal code; the text decoded as ISO-8859-1
 ↪  garbled 5
#    last names; 40.01, 105.27 (no minus sign) came back as a point in China, with weather.
# 3. Boulder's grid square had 115 nights below 0 °C in 2025 and a mean high of 19.7 °C;
 ↪  Leadville's 252
#    and 8.1 °C; Grand Junction's (the key's place) 122 and 18.9 °C. POWER's squares sit at 1,801
 ↪  m
#    (Boulder is about 1,650 m), 3,234 m and 1,904 m (Grand Junction is about 1,400 m): these
 ↪  numbers
#    describe a model's square of land, not a town's thermometer.
# 4. The roster's publish-date (November 3, 2026); POWER's units (C, mm/day), its fill_value
 ↪  (-999.0,
#    on no day of 2025, but on the last 3 of the most recent days), its sources (MERRA2) and the
 ↪  point
#    and elevation it used. Save the publish-date, the request's URL, the units and the point
 ↪  beside the
#    table, so a reader knows which snapshot and which square the numbers describe.
```

Friday: from the lab to the textbook (15 minutes)

What did the lab turn up that the chapter could say better? Some places to look:

- The chapter's "Working with Real XML" names two blocks of the roster we don't read, `<title-info>` and each member's `<committee-assignments>`. Your browser's outline of the file shows a third. What is it, and could `soup.find()` pick up one of its tags by mistake?
- The chapter's printed output says the roster was published September 2, 2026, and our saved copy says November 3. Which of the chapter's counts still hold, and how should a chapter print a number from a file that changes?
- The chapter guards each field with `if m.find(...) else None`. The Clerk's User Guide says which tags every member has, and a vacancy breaks one of its promises. Should the chapter say so, and guard `<footnote>` instead?

- The chapter fetches the roster and the forecast without `headers=`, though week 2 and its own Recommended Exercises ask for a User-Agent. Should its code send one?
- A microsprint that would make a good Additional Exercise: the six non-voting members, the footnotes, the broken JSON, the most recent days, or a place you know.
- A sentence in the chapter that confused you before this week, and one you could now explain to someone else.

`handouts/common/revision-framework.md` says how to propose a revision.